



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика, искусственный интеллект и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Летучка
по курсу «Языки и методы программирования»
«Изучение протокола MQTT»

Студент группы ИУ9-21Б Яннаев А. С.

Преподаватель Посевин Д. П.

Москва 2025

1 Цель работы

Целью лабораторной работы является приобретение навыка разработки на языке JAVA приложения выполняющего запись данных в топик публичного MQTT брокера и приложения чтения данных из топика.

2 Задание

Реализовать на языке JAVA запись и чтение данных в(из) топика в соответствии со своим вариантом, подготовить и прислать отчет, демонстрацию работы программы необходимо записать. Вариант №1

3 Результаты

Исходный код программы представлен в 1, 2, 3, 4, 5, 6, 7.

Листинг 1: Файл

```
1 import org.eclipse.paho.client.mqttv3.MqttClient;
2 import org.eclipse.paho.client.mqttv3.MqttException;
3 import org.eclipse.paho.client.mqttv3.MqttMessage;
4 import java.util.Scanner;
5
6 public class publish {
7     private static final String BROKER = "tcp://test.mosquitto.org:1883";
8     private static final String TOPIC = "IU/9/Yannaev";
9     private static final Scanner input = new Scanner(System.in);
10
11     public static void main(String[] args) {
12         try {
13             MqttClient client = new MqttClient(BROKER, MqttClient.generateClientId());
14             client.connect();
15
16             double a = getNumber("a: ");
17             double b = getNumber("b: ");
18             double c = getNumber("c: ");
19
20             String msg = String.format("%f,%f,%f", a, b, c);
21             client.publish(TOPIC, new MqttMessage(msg.getBytes()));
22             System.out.println("sent");
```

```

23
24         client.disconnect();
25     } catch (MqttException e) {
26         System.out.println("error");
27     }
28 }
29
30 private static double getNumber(String prompt) {
31     while (true) {
32         System.out.print(prompt);
33         try {
34             return Double.parseDouble(input.nextLine());
35         } catch (NumberFormatException e) {
36             System.out.println("bad input");
37         }
38     }
39 }
40 }

```

Листинг 2: Файл

```

1 import org.eclipse.paho.client.mqttv3.*;
2 import org.eclipse.paho.client.mqttv3.persist.MemoryPersistence;
3
4 public class subscribe implements MqttCallback {
5     private static final String BROKER = "tcp://test.mosquitto.org:1883";
6     private static final String TOPIC = "IU/9/Yannaev";
7     private MqttClient client;
8
9     public subscribe() {
10         try {
11             client = new MqttClient(BROKER, MqttClient.generateClientId(), new MemoryPersistence());
12             client.setCallback(this);
13             client.connect();
14             client.subscribe(TOPIC);
15             System.out.println("Connected and subscribed to " + TOPIC);
16         } catch (MqttException e) {
17             System.out.println("Error connecting: " + e.getMessage());
18         }
19     }
20
21     public void connectionLost(Throwable cause) {
22         System.out.println("Connection lost: " + cause.getMessage());
23     }
24 }

```

```

25 public void messageArrived(String topic , MqttMessage message) throws
    Exception {
26     String payload = new String(message.getPayload());
27     System.out.println("Received: " + payload);
28
29     try {
30         String[] parts = payload.split(",");
31         double a = Double.parseDouble(parts[0]);
32         double b = Double.parseDouble(parts[1]);
33         double c = Double.parseDouble(parts[2]);
34
35         if (a == 0) {
36             System.out.println("Not quadratic (a=0)");
37             return;
38         }
39
40         double discriminant = b*b - 4*a*c;
41
42         if (discriminant < 0) {
43             System.out.println("No real roots");
44         } else if (discriminant == 0) {
45             double x = -b / (2*a);
46             System.out.println("x = " + x);
47         } else {
48             double x1 = (-b + Math.sqrt(discriminant)) / (2*a);
49             double x2 = (-b - Math.sqrt(discriminant)) / (2*a);
50             System.out.println("x1 = " + x1 + ", x2 = " + x2);
51         }
52     } catch (Exception e) {
53         System.out.println("Bad format: " + payload);
54     }
55 }
56
57 public void deliveryComplete(IMqttDeliveryToken token) {
58 }
59
60 public static void main(String[] args) {
61     new subscribe();
62 }
63 }

```

Листинг 3: Файл

```

1 import javax.swing.*;
2 import java.awt.event.ActionEvent;
3 import java.awt.event.ActionListener;
4 import java.io.FileWriter;

```

```

5
6 public class publish {
7     public JPanel mainPanel;
8     public JTextField aField;
9     public JTextField bField;
10    public JTextField cField;
11    public JButton sendButton;
12
13    public publish() {
14        sendButton.addActionListener(new ActionListener() {
15            public void actionPerformed(ActionEvent e) {
16                try {
17                    double a = Double.parseDouble(aField.getText());
18                    double b = Double.parseDouble(bField.getText());
19                    double c = Double.parseDouble(cField.getText());
20                    String data = a + "," + b + "," + c;
21
22                    FileWriter fw = new FileWriter("/home/keiichi/
IdeaProjects/wth_is_this/src/2nd/publish/publish.txt");
23                    fw.write(data);
24                    fw.close();
25
26                    JOptionPane.showMessageDialog(mainPanel, "Sent: " +
data);
27                } catch (Exception ex) {
28                    JOptionPane.showMessageDialog(mainPanel, "Wrong
input");
29                }
30            }
31        });
32    }
33
34    public static void main(String[] args) {
35        JFrame frame = new JFrame("MQTT Publisher - Yannaev");
36        frame.setContentPane(new publish().mainPanel);
37        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
38        frame.pack();
39        frame.setVisible(true);
40    }
41 }

```

Листинг 4: Файл

```

1 import org.eclipse.paho.client.mqttv3.MqttClient;
2 import org.eclipse.paho.client.mqttv3.MqttException;
3 import org.eclipse.paho.client.mqttv3.MqttMessage;
4 import java.io.*;

```

```

5 import java.nio.file.*;
6
7 public class publish {
8     private static final String BROKER = "tcp://test.mosquitto.org:1883"
9     ;
10    private static final String TOPIC = "IU/9/Yannaev";
11    private static final String FILE_PATH = "/home/keiichi/IdeaProjects/
with_is_this/src/2nd/publish/publish.txt";
12
13    private static boolean running = true;
14
15    public static void main(String[] args) {
16        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
17            running = false;
18            System.out.println("stopped");
19        }));
20
21        try {
22            MqttClient client = new MqttClient(BROKER, MqttClient.
generateClientId());
23            client.connect();
24            System.out.println("waiting for input in " + FILE_PATH);
25
26            while (running) {
27                try {
28                    if (Files.exists(Paths.get(FILE_PATH))) {
29                        String content = new String(Files.readAllBytes(
Paths.get(FILE_PATH))).trim();
30                        if (!content.isEmpty()) {
31                            try {
32                                String[] parts = content.split(",");
33                                double a = Double.parseDouble(parts[0]);
34                                double b = Double.parseDouble(parts[1]);
35                                double c = Double.parseDouble(parts[2]);
36                                String msg = a + "," + b + "," + c;
37                                client.publish(TOPIC, new MqttMessage(
msg.getBytes()));
38                                System.out.println("sent: " + msg);
39                                Files.write(Paths.get(FILE_PATH), "".
getBytes());
40                            } catch (Exception e) {
41                                System.out.println("bad format, need: a,
b,c");
42                            }
43                        }
44                    }
45                }
46            }
47            Thread.sleep(1000);

```

```

44         } catch (InterruptedException e) {
45             Thread.currentThread().interrupt();
46         } catch (IOException e) {
47             System.out.println("Error reading file: " + e.
getMessage());
48         }
49     }
50     client.disconnect();
51 } catch (MqttException e) {
52     System.out.println("MQTT error: " + e.getMessage());
53 }
54 }
55 }

```

Листинг 5: Файл

```

1 import org.eclipse.paho.client.mqttv3.*;
2 import org.eclipse.paho.client.mqttv3.persist.MemoryPersistence;
3 import java.io.FileWriter;
4 import java.io.IOException;
5 import java.text.DecimalFormat;
6
7 public class subscribe implements MqttCallback {
8     private static final String BROKER = "tcp://test.mosquitto.org:1883"
;
9     private static final String TOPIC = "IU/9/Yannaev";
10    private static final String FILE_PATH = "/home/keiichi/IdeaProjects/
wth_is_this/src/2nd/subscribe/subscribe.txt";
11    private MqttClient client;
12    private DecimalFormat df = new DecimalFormat("0.00");
13
14    public subscribe() {
15        try {
16            client = new MqttClient(BROKER, MqttClient.generateClientId
(), new MemoryPersistence());
17            client.setCallback(this);
18            client.connect();
19            client.subscribe(TOPIC);
20            System.out.println("Subscribed to " + TOPIC + ", writing
solutions to " + FILE_PATH);
21        } catch (MqttException e) {
22            System.out.println("Error connecting: " + e.getMessage());
23        }
24    }
25
26    private String solveQuadratic(double a, double b, double c) {
27        if (a == 0) {

```

```

28         return "Error: a=0 (not quadratic)";
29     }
30
31     double discriminant = b*b - 4*a*c;
32     if (discriminant < 0) {
33         return "No real roots";
34     } else if (discriminant == 0) {
35         double x = -b / (2*a);
36         return "x=" + df.format(x);
37     } else {
38         double x1 = (-b + Math.sqrt(discriminant)) / (2*a);
39         double x2 = (-b - Math.sqrt(discriminant)) / (2*a);
40         return "x1=" + df.format(x1) + ";x2=" + df.format(x2);
41     }
42 }
43
44 public void connectionLost(Throwable cause) {
45     System.out.println("Connection lost: " + cause.getMessage());
46 }
47
48 public void messageArrived(String topic, MqttMessage message) {
49     try {
50         String payload = new String(message.getPayload());
51         System.out.println("Received: " + payload);
52
53         String[] parts = payload.split(",");
54         double a = Double.parseDouble(parts[0]);
55         double b = Double.parseDouble(parts[1]);
56         double c = Double.parseDouble(parts[2]);
57
58         String solution = solveQuadratic(a, b, c);
59         writeToFile(solution);
60         System.out.println("Solved " + a + " x " + b + "x" + c + "
61 =0: " + solution);
62     } catch (NumberFormatException e) {
63         writeToFile("Error: bad input format");
64         System.out.println("Error: expected 'a,b,c' numbers");
65     } catch (Exception e) {
66         writeToFile("Error: " + e.getMessage());
67         System.out.println("Error: " + e.getMessage());
68     }
69 }
70
71 private void writeToFile(String content) {
72     try (FileWriter writer = new FileWriter(FILE_PATH)) {

```



```

73         writer.write(content);
74     } catch (IOException e) {
75         System.out.println("Error writing to file: " + e.getMessage
76     );
77     }
78 }
79
80 public void deliveryComplete(IMqttDeliveryToken token) {}
81
82 public static void main(String[] args) {
83     new subscribe();
84
85     // Keep the program running
86     while (true) {
87         try {
88             Thread.sleep(1000);
89         } catch (InterruptedException e) {
90             Thread.currentThread().interrupt();
91             break;
92         }
93     }
94 }

```

Листинг 6: Файл

```

1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.ActionEvent;
4 import java.awt.event.ActionListener;
5 import java.io.BufferedReader;
6 import java.io.FileReader;
7
8 public class subscribe {
9     private JPanel mainPanel;
10    private JButton refreshButton;
11    private JLabel solutionLabel;
12    private static final String FILE_PATH = "/home/keiichi/IdeaProjects/
13    wth_is_this/src/2nd/subscribe/subscribe.txt";
14
15    public subscribe() {
16        solutionLabel.setFont(new Font("Monospaced", Font.PLAIN, 14));
17        solutionLabel.setHorizontalAlignment(SwingConstants.CENTER);
18
19        loadSolution();
20
21        refreshButton.addActionListener(new ActionListener() {

```

```

21         public void actionPerformed(ActionEvent e) {
22             loadSolution();
23         }
24     });
25 }
26
27 private void loadSolution() {
28     try (BufferedReader reader = new BufferedReader(new FileReader(
29 FILE_PATH))) {
30         String solution = reader.readLine();
31         if (solution != null && !solution.isEmpty()) {
32             solutionLabel.setText(solution);
33         } else {
34             solutionLabel.setText("No solution available");
35         }
36     } catch (Exception e) {
37         solutionLabel.setText("Error reading solution");
38     }
39 }
40
41 public static void main(String[] args) {
42     SwingUtilities.invokeLater(() -> {
43         JFrame frame = new JFrame("Equation Solver");
44         frame.setContentPane(new subscribe().mainPanel);
45         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46         frame.setSize(300, 150);
47         frame.setMinimumSize(new Dimension(250, 120));
48         frame.setLocationRelativeTo(null);
49         frame.setVisible(true);
50     });
51 }

```

Листинг 7: Файл

```

1 import org.eclipse.paho.client.mqttv3.*;
2 import javax.swing.*;
3 import java.awt.*;
4 import java.awt.event.ActionEvent;
5 import java.util.Date;
6
7 public class MQTTChat implements MqttCallback {
8     private JPanel mainPanel;
9     private JTextArea chatArea;
10    private JTextField messageField;
11    private JButton sendButton;
12    private JScrollPane scrollPane;

```

```

13
14     private static final String BROKER = "tcp://test.mosquitto.org:1883"
15     ;
16     private static final String TOPIC = "/iu9/mqttchat";
17     private static final String CLIENT_ID = MqttClient.generateClientId
18     ();
19     private MqttClient client;
20     private String username;
21
22     public MQTTChat() {
23         setupUI();
24         setupMQTT();
25     }
26
27     private void setupUI() {
28         //
29         sendButton.addActionListener(this::sendMessage);
30         messageField.addActionListener(this::sendMessage);
31
32         //
33         chatArea.setEditable(false);
34         chatArea.setFont(new Font("Arial", Font.PLAIN, 14));
35         chatArea.setBackground(new Color(240, 240, 240));
36         mainPanel.setBorder(BorderFactory.createEmptyBorder(5, 5, 5, 5))
37     ;
38     }
39
40     private void setupMQTT() {
41         try {
42             //
43             username = JOptionPane.showInputDialog(
44                 mainPanel,
45                 "Enter your username:",
46                 "Login",
47                 JOptionPane.PLAIN_MESSAGE
48             );
49
50             if (username == null || username.trim().isEmpty()) {
51                 username = "Anonymous_" + (int)(Math.random() * 1000);
52             }
53
54             client = new MqttClient(BROKER, CLIENT_ID);
55             client.setCallback(this);
56             client.connect();
57             client.subscribe(TOPIC);
58             addMessage("System", "Connected to chat as " + username);

```

```

56         } catch (MqttException e) {
57             JOptionPane.showMessageDialog(
58                 mainPanel,
59                 "Failed to connect: " + e.getMessage(),
60                 "Error",
61                 JOptionPane.ERROR_MESSAGE
62             );
63             System.exit(1);
64         }
65     }
66
67     private void sendMessage(ActionEvent e) {
68         String text = messageField.getText().trim();
69         if (!text.isEmpty()) {
70             try {
71                 String message = username + ": " + text;
72                 client.publish(TOPIC, new MqttMessage(message.getBytes())
73             ));
74                 messageField.setText("");
75             } catch (MqttException ex) {
76                 addMessage("System", "Failed to send message");
77             }
78         }
79
80     public void messageArrived(String topic, MqttMessage message) {
81         String text = new String(message.getPayload());
82         SwingUtilities.invokeLater(() -> {
83             String[] parts = text.split(":", 2);
84             if (parts.length == 2) {
85                 addMessage(parts[0].trim(), parts[1].trim());
86             } else {
87                 addMessage("System", text);
88             }
89         });
90     }
91
92     private void addMessage(String sender, String message) {
93         String formatted = String.format("[%tT] %s: %s\n", new Date(),
94 sender, message);
95         chatArea.append(formatted);
96         chatArea.setCaretPosition(chatArea.getDocument().getLength());
97
98         if (sender.equals(username)) {
99             int start = chatArea.getText().length() - formatted.length()
100 ;

```

```

99         int end = chatArea.getText().length();
100         chatArea.select(start, end);
101         chatArea.setSelectionColor(new Color(220, 240, 255));
102     }
103 }
104
105 public void connectionLost(Throwable cause) {
106     SwingUtilities.invokeLater(() ->
107         addMessage("System", "Connection lost: " + cause.
108         getMessage()));
109 }
110
111 public void deliveryComplete(IMqttDeliveryToken token) {}
112
113 public static void main(String[] args) {
114     SwingUtilities.invokeLater(() -> {
115         JFrame frame = new JFrame("MQTT Chat");
116         frame.setContentPane(new MQTTChat().mainPanel);
117         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
118         frame.setSize(600, 400);
119         frame.setLocationRelativeTo(null);
120         frame.setVisible(true);
121     });
122 }

```

Результат запуска представлен на рисунке 1.

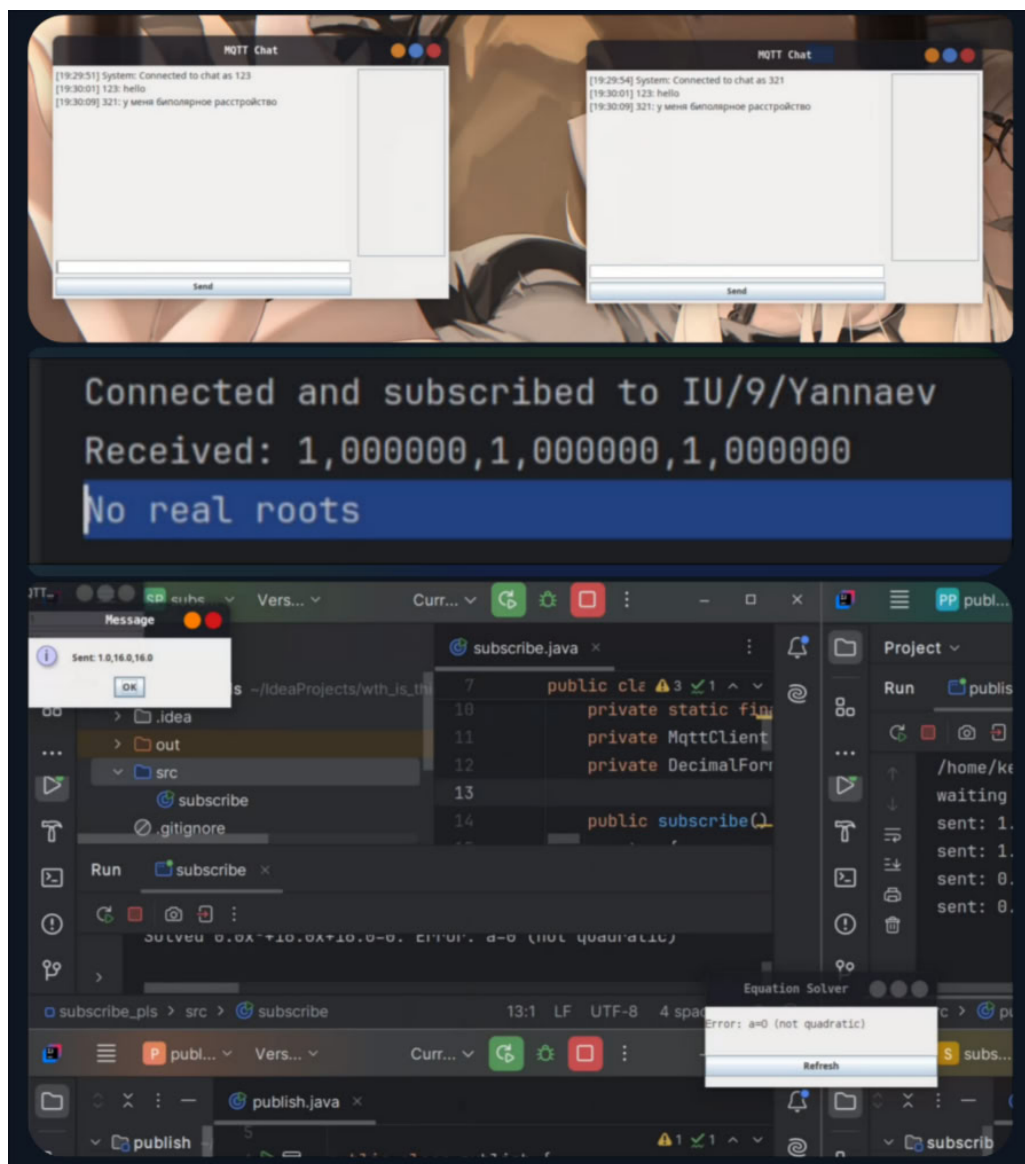


Рис. 1 — Результат